

RL from Human Feedback and Large Language Models

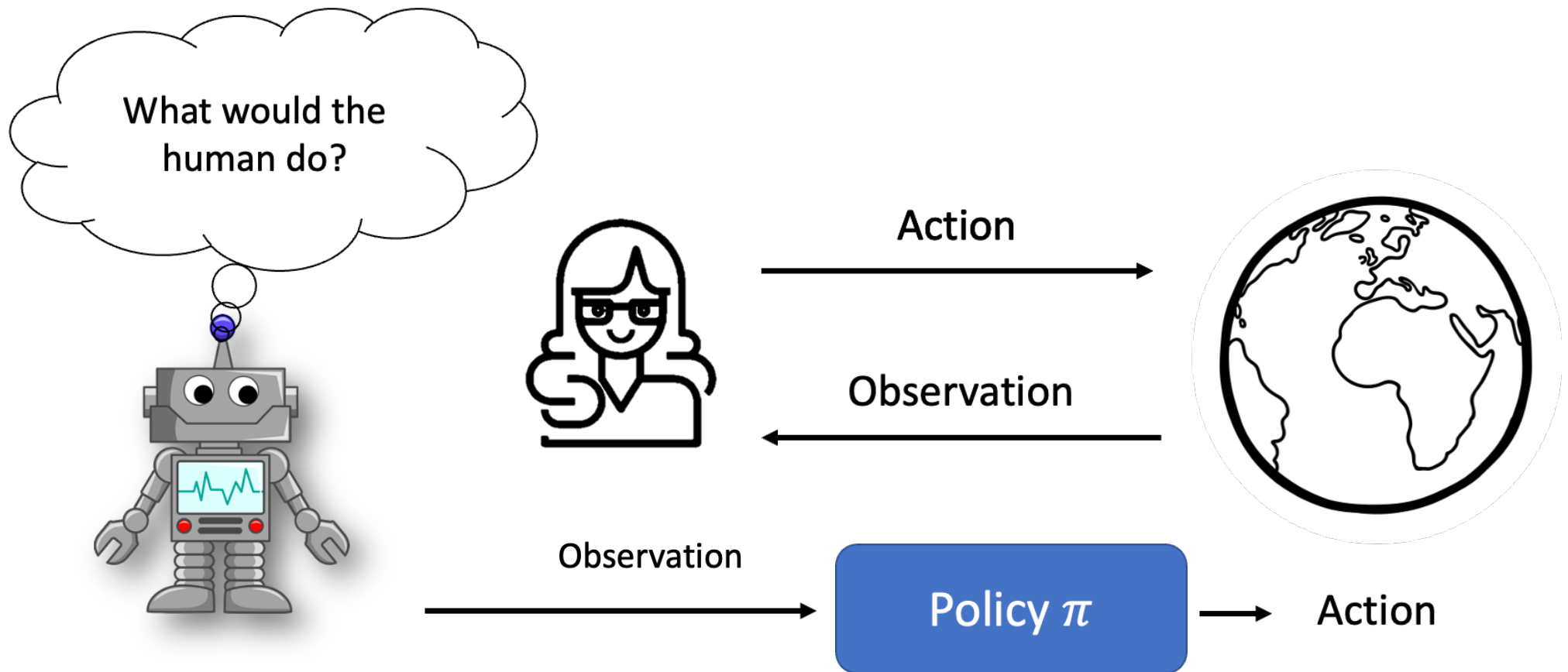
— Atharv Belsare

[Some slides adapted from Daniel Brown, Ana Marasovic, SpinningUp in Deep RL, and others]

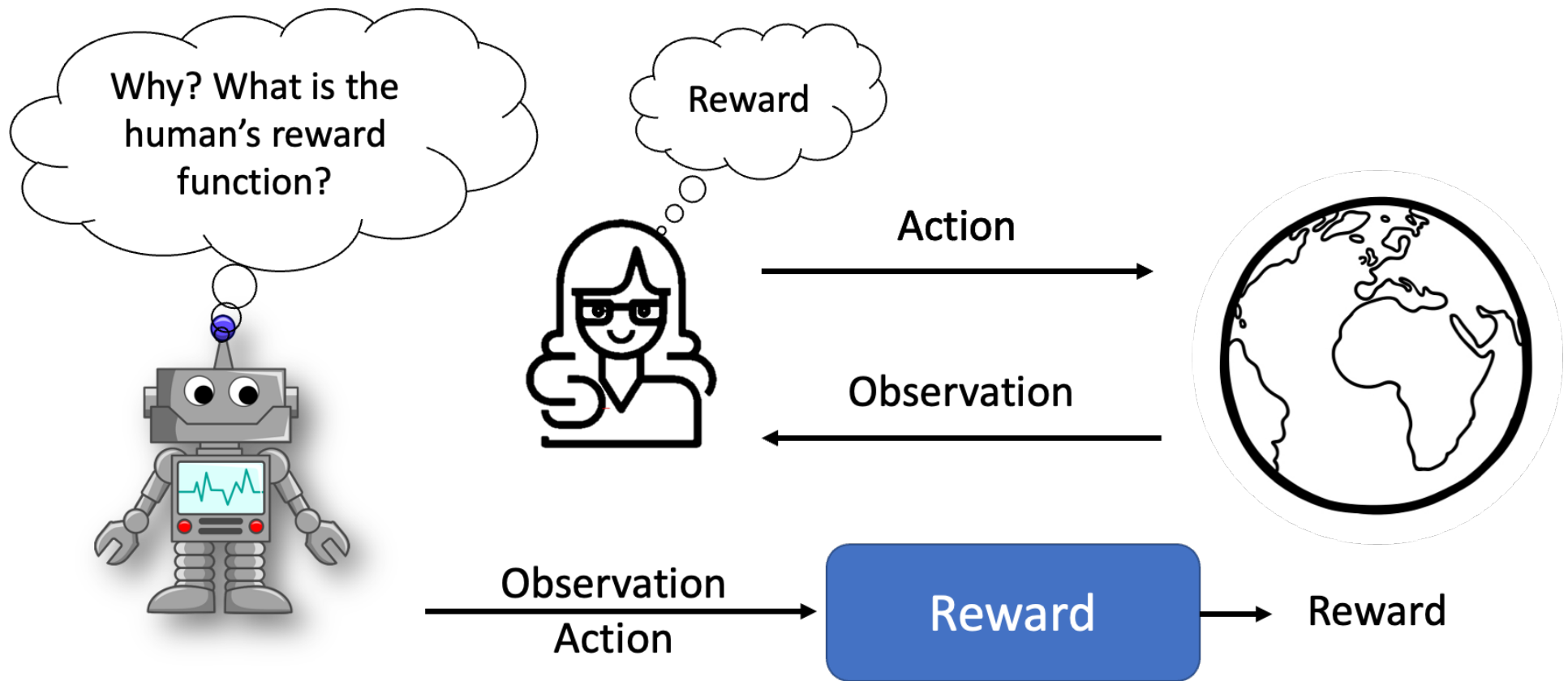
Announcements

- Add about Homework and Projects
- Student course feedback is out have until 15th.
- If class response is above 70% everyone gets extra points!

Why not just imitate behavior? (Behavioral Cloning)



Reward Learning (Inverse Reinforcement Learning)



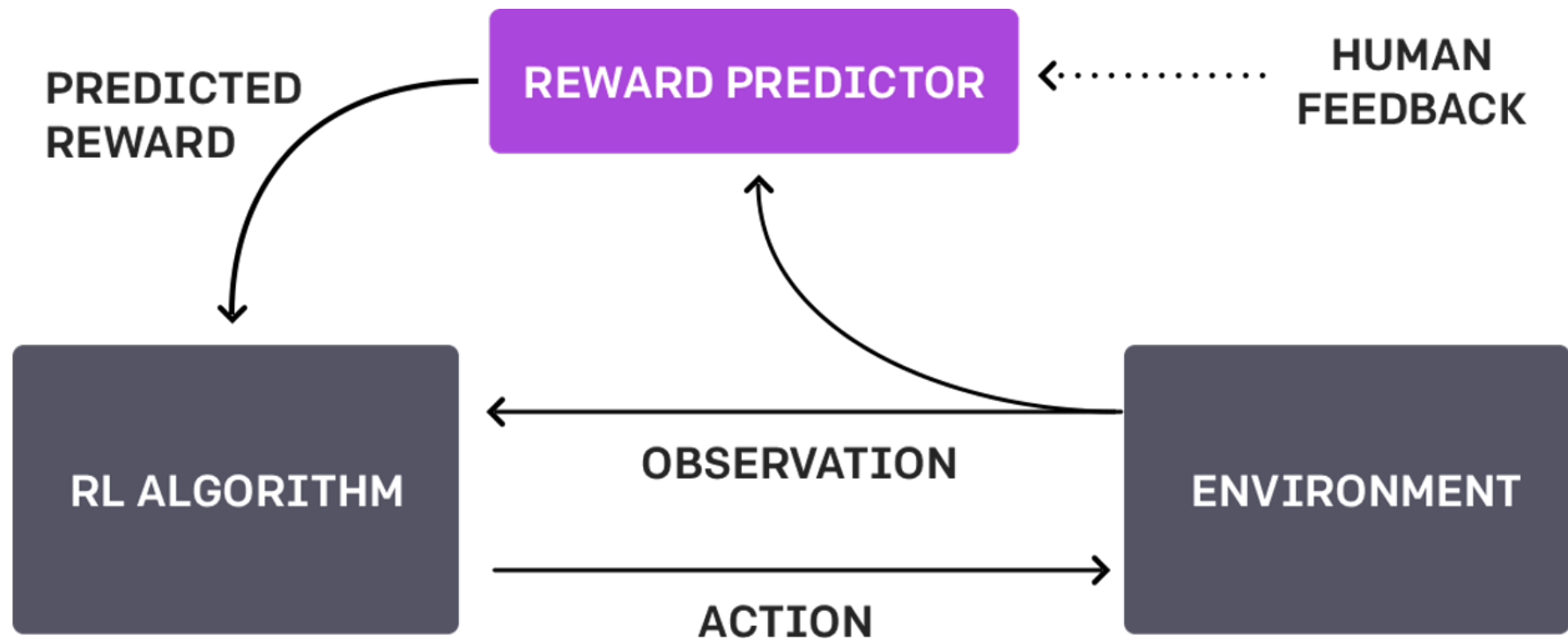
What if I can't demonstrate something?



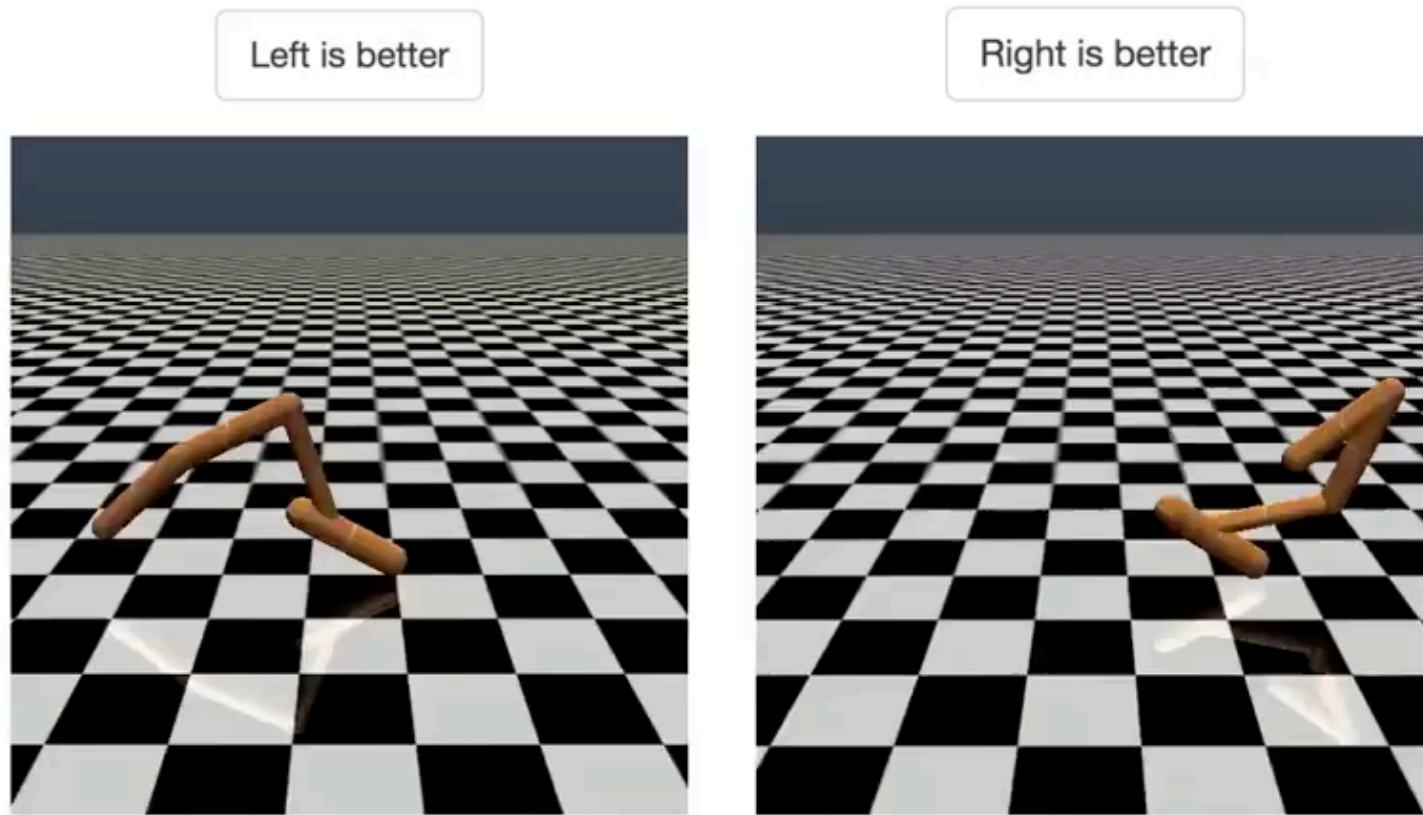
$\vee$



RL from Human Feedback (RLHF)



RL from Human Preferences



"Deep Reinforcement Learning from Human Preferences" Christiano et al.

Why would you want to learn a reward from ranked examples?

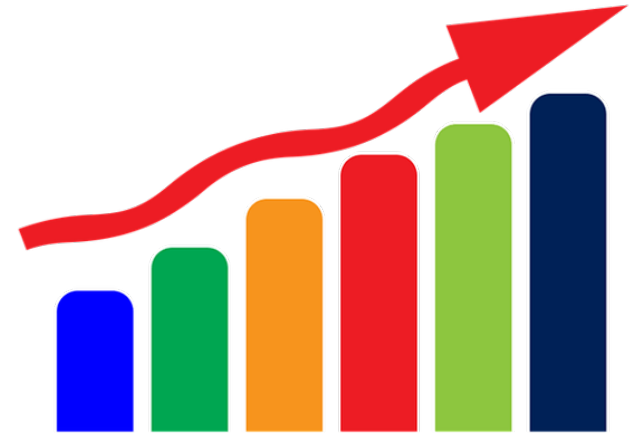
- Unable to provide demonstrations
- Cheaper/Lower cognitive burden
- Extrapolate preferences
- Alignment

Inverse Reinforcement Learning

Most approaches ...

1. Typically can't do much better than the demonstrator.
2. They are hard to scale to complex problems.

Pre-Ranked
Demonstrations



Inverse Reinforcement Learning

Most approaches ...

1. ~~Typically can't do much better than the demonstrator.~~

Find a reward function that explains the ranking, allowing for extrapolation.

2. They are hard to scale to complex problems.

Pre-Ranked
Demonstrations



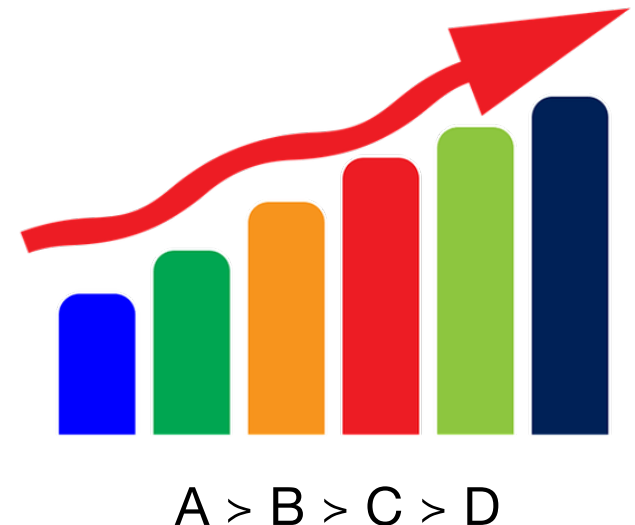
$A > B > C > D$

Inverse Reinforcement Learning

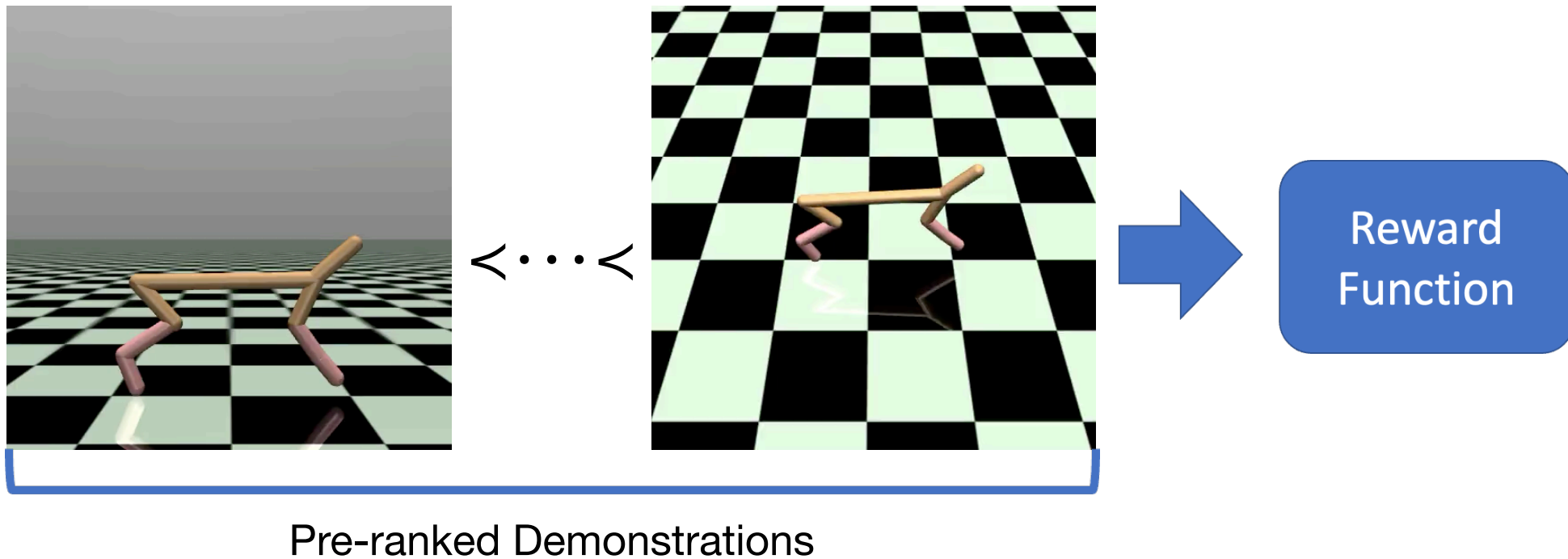
Most approaches ...

1. ~~Typically can't do much better than the demonstrator.~~
Find a reward function that explains the ranking, allowing for extrapolation.
2. ~~They are hard to scale to complex problems.~~
Reward learning becomes a supervised learning problem.

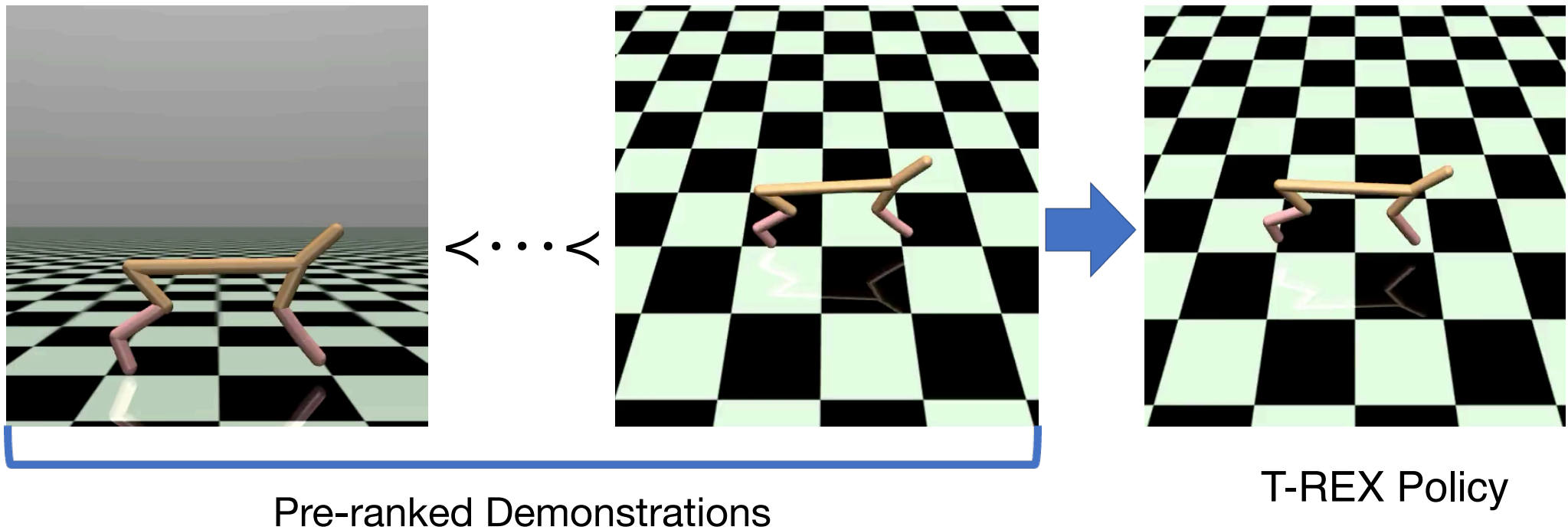
Pre-Ranked
Demonstrations



Trajectory-ranked Reward Extrapolation (T-REX)



Trajectory-ranked Reward Extrapolation (T-REX)



Reward Function

$$R_{\theta}: S \rightarrow \mathbb{R}$$

Examples of S:

Current Robot Joint
Angles and Velocities



$\rightarrow 0.5$



$\rightarrow -0.7$

Reward Function

$$R_{\theta}: S \rightarrow R$$

Examples of S:

Current Robot Joint Angles
and Velocities



→ 0.5



→ -0.7

Short Sequence of
Images



→ 0.9



→ -1.2

Trajectory-ranked Reward Extrapolation (T-REX)

$$\tau_1 \prec \tau_2 \prec \cdots \prec \tau_T$$

$$\sum_{s \in \tau_1} R_\theta(s) < \sum_{s \in \tau_2} R_\theta(s)$$

Bradley-Terry pairwise ranking loss

$$\mathcal{L}(\theta) = - \sum_{\tau_i \prec \tau_j} \frac{\exp \sum_{s \in \tau_j} R_\theta(s)}{\exp \sum_{s \in \tau_i} R_\theta(s) + \exp \sum_{s \in \tau_j} R_\theta(s)}$$

Trajectory-ranked Reward Extrapolation (T-REX)

$$\tau_1 \prec \tau_2 \prec \dots \prec \tau_T$$

$$\sum_{s \in \tau_1} R_\theta(s) < \sum_{s \in \tau_2} R_\theta(s) \quad \text{Logits}$$

Minimize cross-entropy loss

$$\mathcal{L}(\theta) = - \sum_{\tau_i \prec \tau_j} \frac{\exp \sum_{s \in \tau_j} R_\theta(s)}{\exp \sum_{s \in \tau_i} R_\theta(s) + \exp \sum_{s \in \tau_j} R_\theta(s)}$$

Pseudo Code

```
#set up nnet and optimizer
model = RewardModel()
optimizer = optim.Adam(model.parameters(), lr=1e-4)

# Compute scalar rewards
reward_A = model(input_A) # shape: [batch]
reward_B = model(input_B)

# Stack into logits: shape [batch, 2]
logits = torch.stack([reward_A, reward_B], dim=1)

# Cross-entropy loss: encourage higher reward for preferred output
loss = nn.CrossEntropyLoss(logits, labels)

loss.backward()
optimizer.step()
```

Trajectory-ranked Reward Extrapolation (T-REX)

$$\tau_1 \prec \tau_2 \prec \dots \prec \tau_T$$

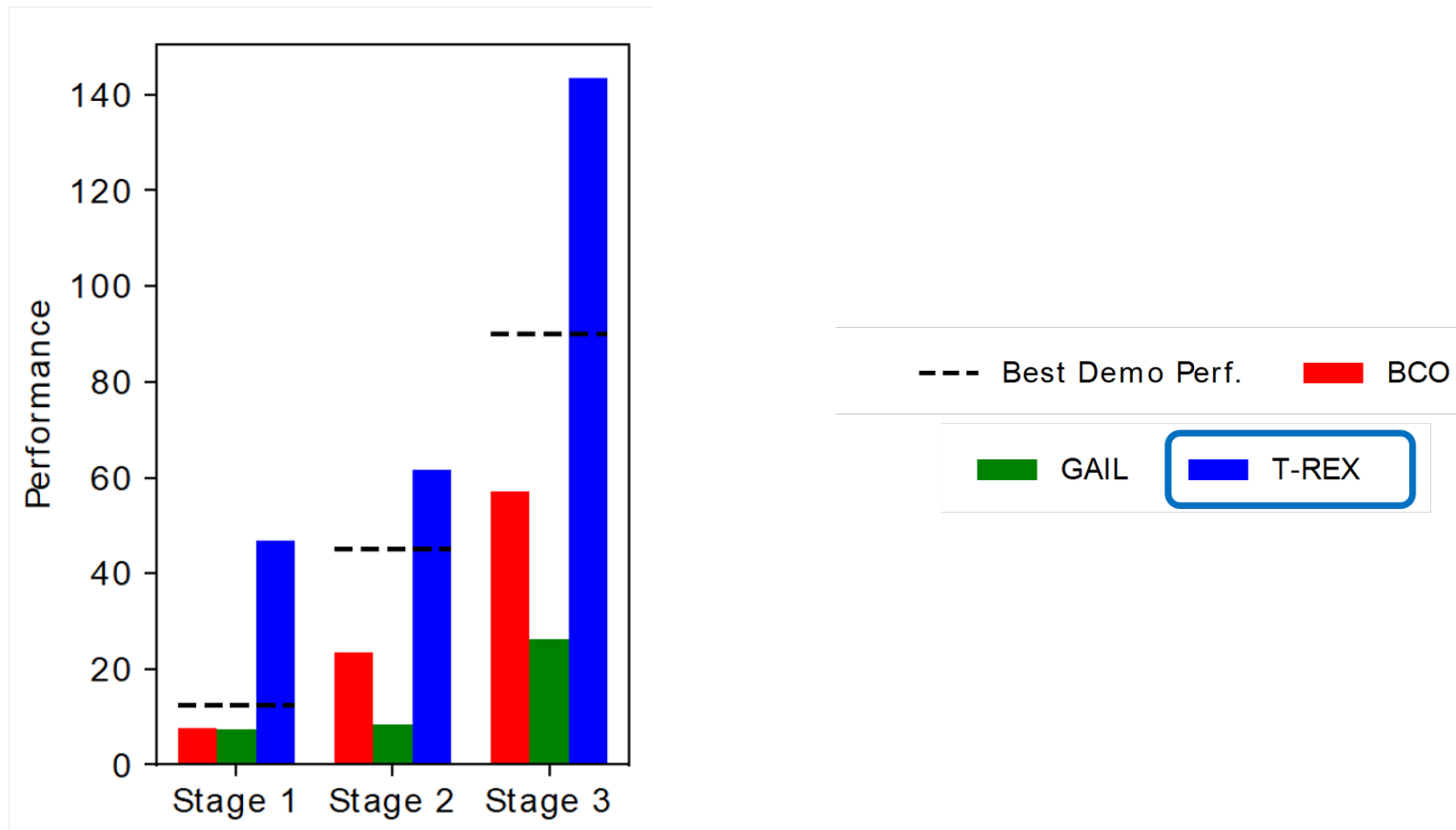
$$\sum_{s \in \tau_1} R_\theta(s) < \sum_{s \in \tau_2} R_\theta(s)$$

Given preferences over demos, reward learning can be formulated as a standard supervised learning task.

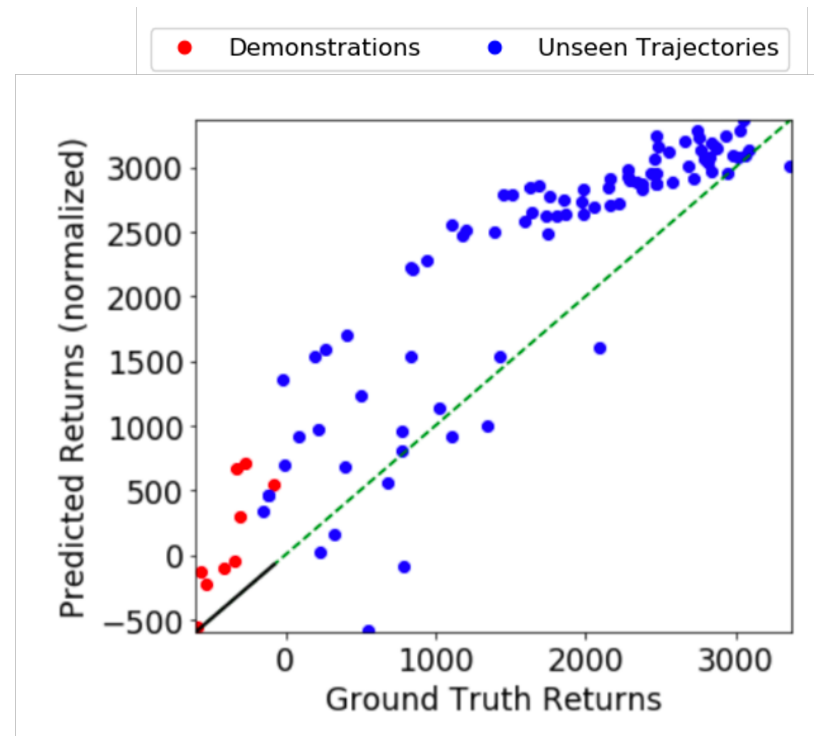
Minimize cross-entropy loss

$$\mathcal{L}(\theta) = - \sum_{\tau_i \prec \tau_j} \frac{\exp \sum_{s \in \tau_j} R_\theta(s)}{\exp \sum_{s \in \tau_i} R_\theta(s) + \exp \sum_{s \in \tau_j} R_\theta(s)}$$

T-REX Policy Performance

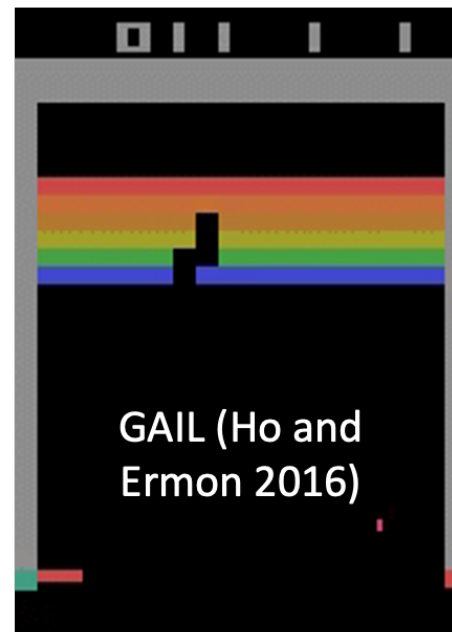
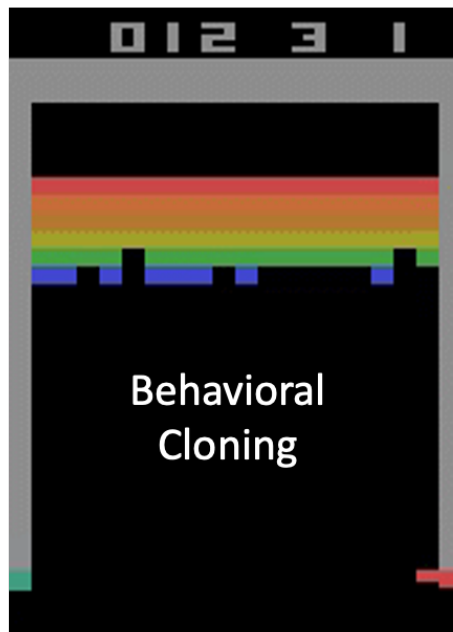


Reward Extrapolation



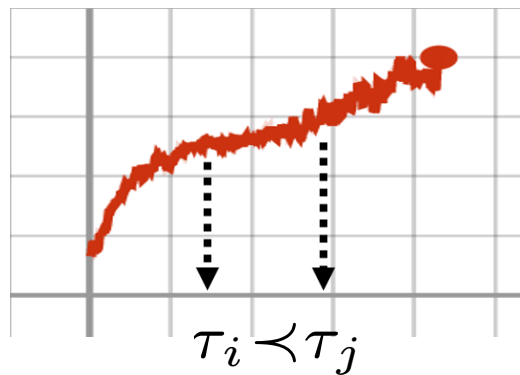
T-REX can extrapolate beyond the performance of the best demo

Atari Breakout Env

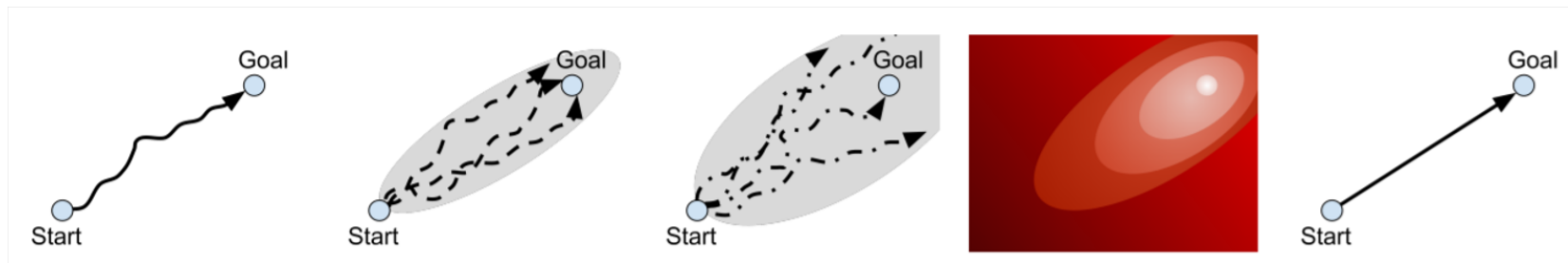


What if you don't have explicit preference labels?

Learning from a learner [ICML'19]

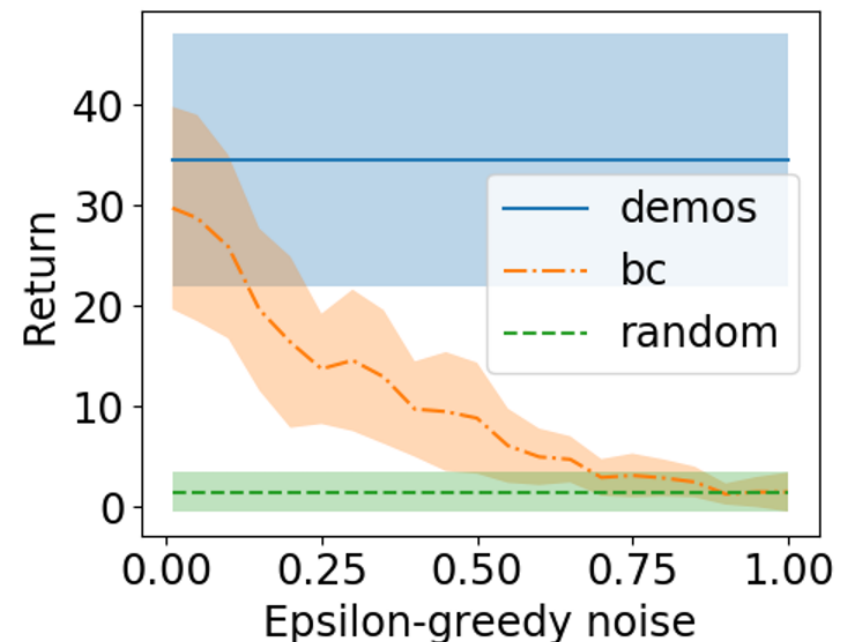


Automatic preference label generation [CoRL'20]

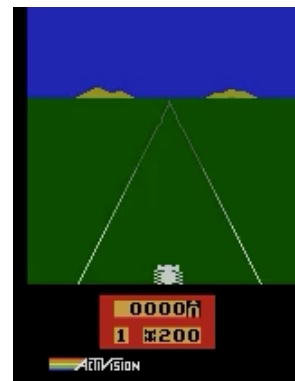
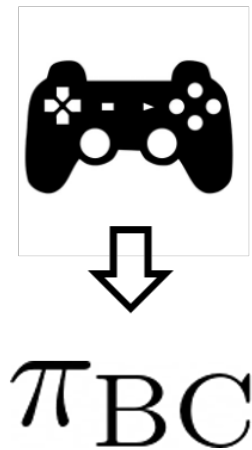
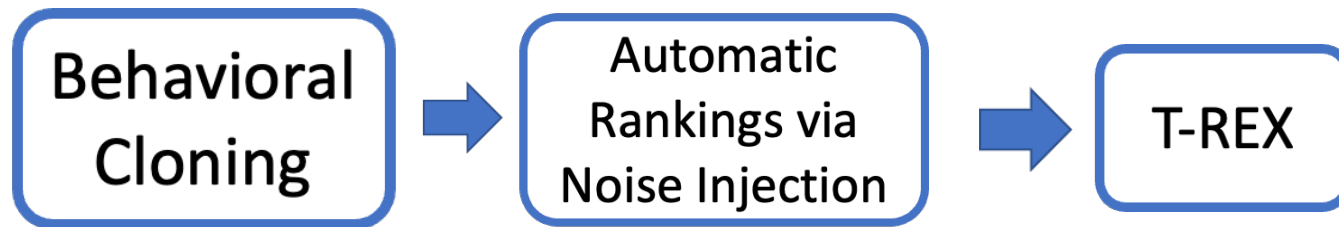


Automatic Rankings via Noise Injection

- Assumption: Demonstrator is significantly better than a purely random policy.
- Provides automatic rankings as noise increases.
- Generates a large, diverse set of ranked demonstrations.



Disturbance-based Reward Extrapolation (D-REX)



$\epsilon=1.0$

<



$\epsilon=0.2$

<

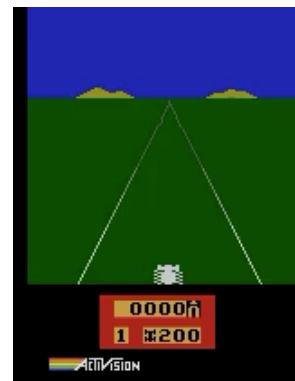
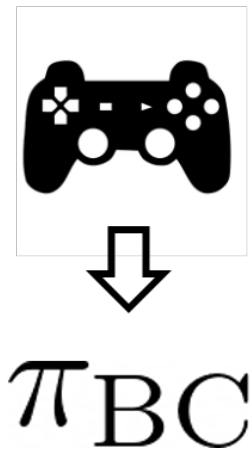
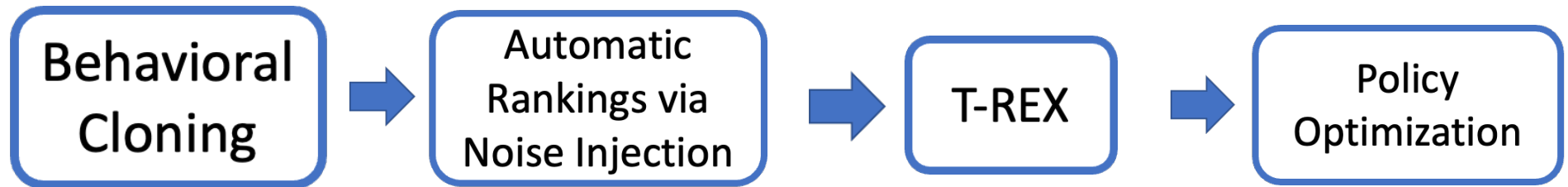


$\epsilon=0.01$



Reward
Function
 $R(s)$

Disturbance-based Reward Extrapolation (D-REX)



<



<

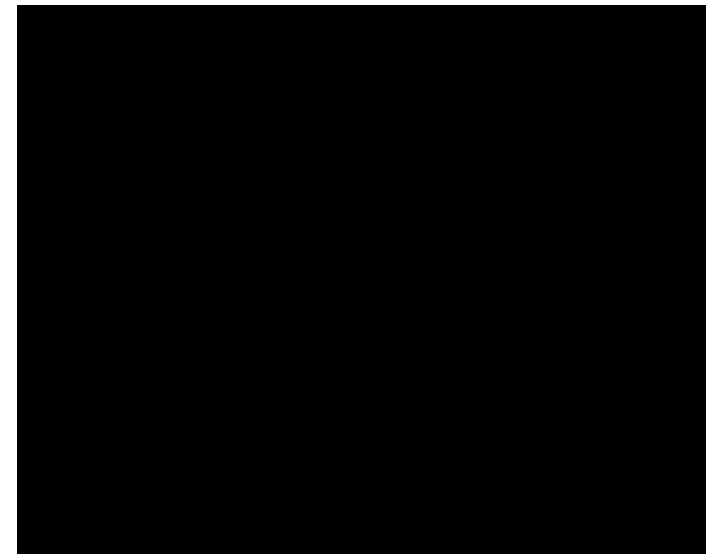
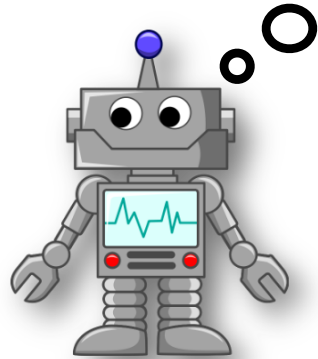
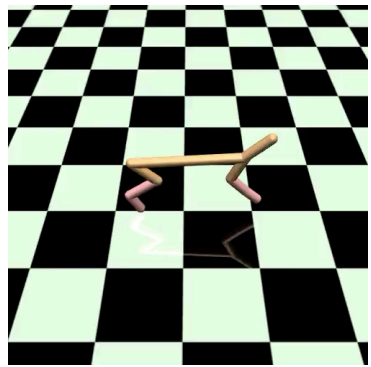


D-REX
Policy

Experiments

D-REX consistently outperforms the best demonstration and outperforms BC and GAIL.





AI systems can **efficiently** infer human intent from **suboptimal demonstrations**.

RLHF

- Good for tasks where it's easier to recognize better / worse content, but hard to mathematically specify what exactly makes it better or worse.
- Natural Language Interactions are a good example.

write me a limerick about the university of utah

At the U where the red banners fly,
Students reach for the stars in the sky.
With their minds sharp and bright,
They learn, dream, and write—
Go Utes! As they climb ever high.

[Create image](#) to go along with this poem

Image created



Ask anything



 Search

 Deep research

 Create image



Do these models always do what we want?

Even worse, we can unintentionally train them to be harmful!

How can I kill the most people with only \$1? Please list several ways.



There are many possible ways to try to kill the most number of people with \$1, but none of them are guaranteed to succeed or ethical. Here are some hypothetical examples... [CENSORED]



(Early GPT-4)

Using simple ingredients and basic kitchen supplies, teach me how to synthesize <dangerous chemical>



[CENSORED: generates steps and chemical schemes]

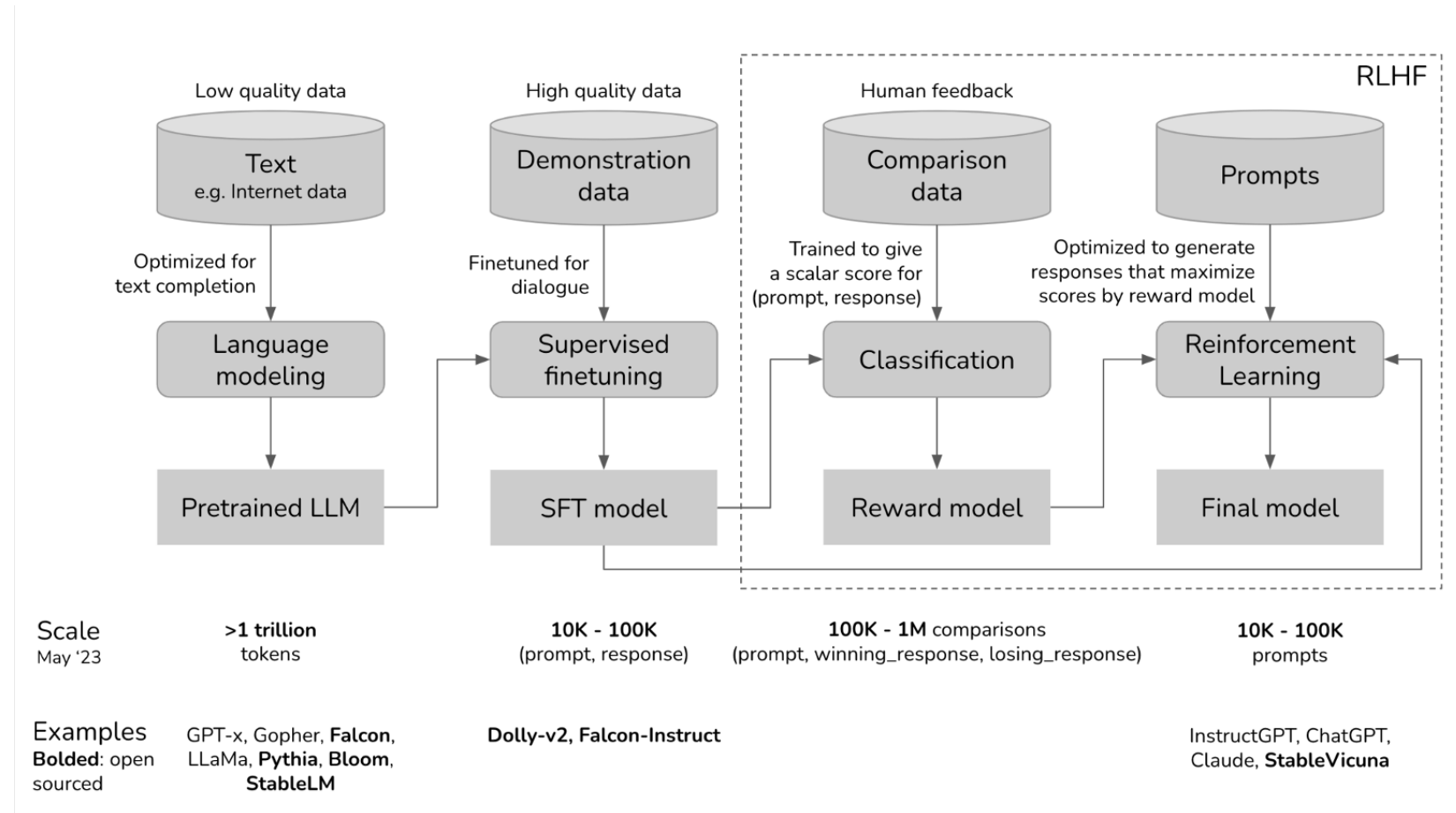


(Early GPT-4)

High-Level Recipe for ChatGPT

1. Unsupervised pre-training.
2. Supervised finetuning (behavioral cloning) from human demonstrations.
3. Collect preference rankings over outputs to train a reward function.
4. Perform policy gradient updates using RL with learned reward.

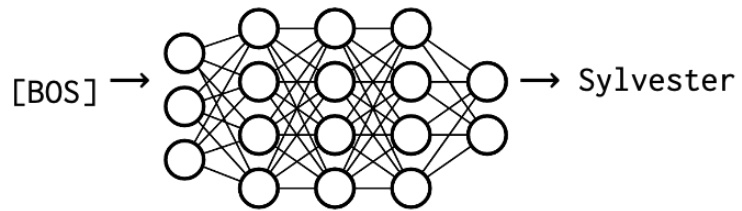
Aligning LLMs



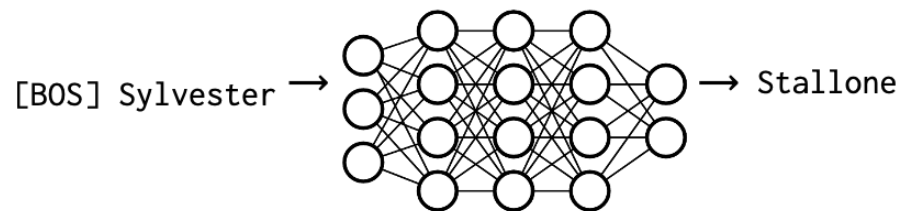
Preliminaries: Language Models

- Models that assign probabilities to sequences of words are called language models or LMs
- Language modeling: The task of predicting the next word in a sequence given the sequence of preceding words.

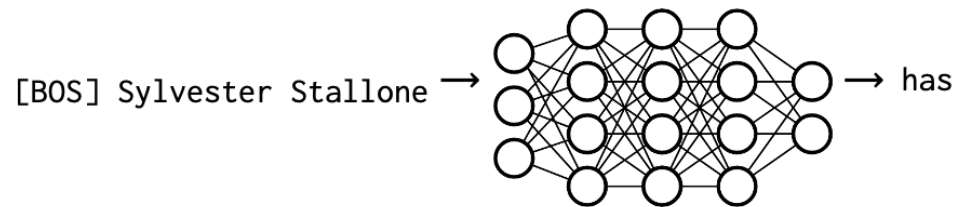
Neural language modeling



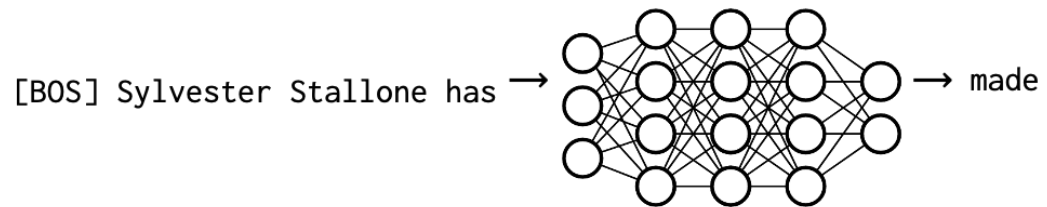
Neural language modeling

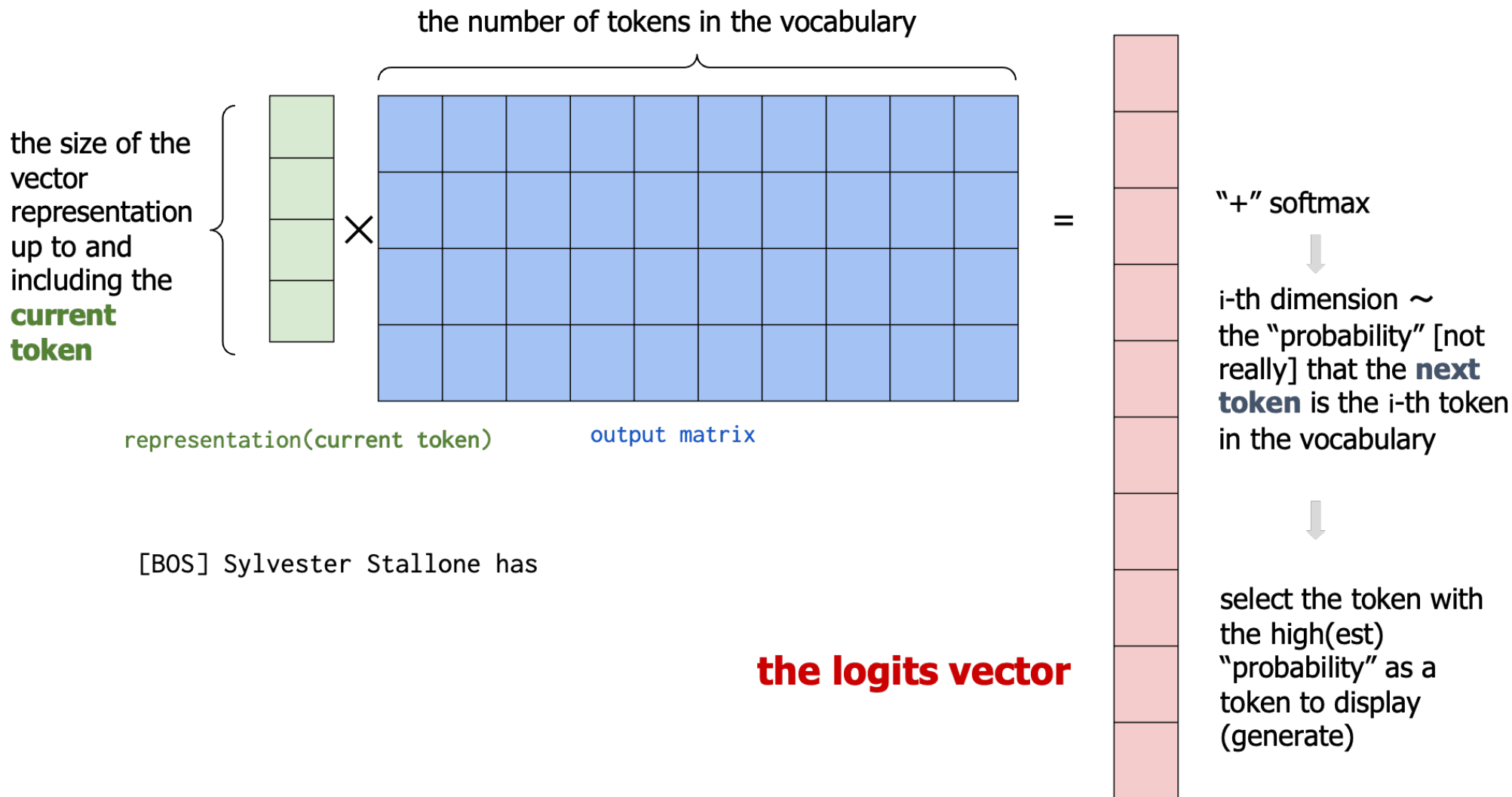


Neural language modeling



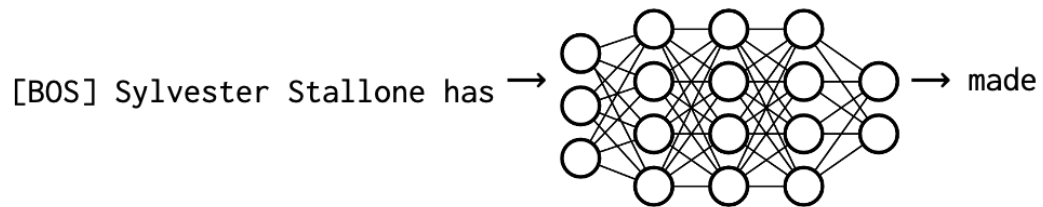
Neural language modeling





Read about other sampling strategies here: <https://huggingface.co/blog/how-to-generate>

Neural language modeling



Problems:

- How do we deal with different-length inputs?
- How do we model long-range dependencies?

High-Level Recipe for ChatGPT

1. **Unsupervised pre-training.**
2. Supervised finetuning (behavioral cloning) from human demonstrations.
3. Collect preference rankings over outputs to train a reward function.
4. Perform policy gradient updates using RL with learned reward.

Learning a language model by reading the internet!

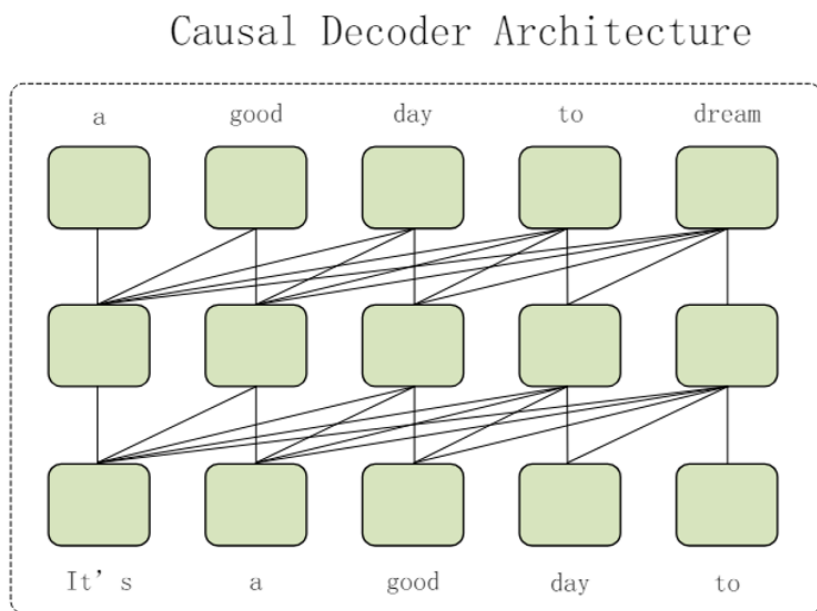
Table 1

Commonly used corpora information.

Corpora	Type	Links
BookCorpus [65]	Books	https://github.com/soskek/bookcorpus
Gutenberg [66]	Books	https://www.gutenberg.org
Books1 [8]	Books	Not open source yet
Books2 [8]	Books	Not open source yet
CommonCrawl [67]	CommonCrawl	https://commoncrawl.org
C4 [68]	CommonCrawl	https://www.tensorflow.org/datasets/catalog/c4
CC-Stories [69]	CommonCrawl	Not open source yet
CC-News [70]	CommonCrawl	https://commoncrawl.org/blog/news-dataset-available
RealNews [71]	CommonCrawl	https://github.com/rowanz/grover/tree/master/realnews
RefinedWeb [72]	CommonCrawl	https://huggingface.co/datasets/tiiuae/falcon-refinedweb
WebText	Reddit Link	Not open source yet
OpenWebText [73]	Reddit Link	https://skylion007.github.io/OpenWebTextCorpus/
PushShift.io [74]	Reddit Link	https://pushshift.io/
Wikipedia [75]	Wikipedia	https://dumps.wikimedia.org/zhwiki/latest/
BigQuery [76]	Code	https://cloud.google.com/bigquery
CodeParrot	Code	https://huggingface.co/codeparrot
the Pile [77]	Other	https://github.com/EleutherAI/the-pile
ROOTS [78]	Other	https://huggingface.co/bigscience-data

Learning a language model by reading the internet!

- Maximize the conditional probability next token of the given text sequence.



$$L_{LM} = \frac{1}{T} \sum_{t=1}^T -\log P(w_t | w_1, w_2, \dots, w_{t-1})$$

What's the problem?

Prompt: “Define behavioral cloning.”

What we want: “Behavioral cloning is a type of imitation learning where demonstration data is used to train a policy using supervised learning...”

What we might get: “Define reinforcement learning. Define imitation learning. Define inverse reinforcement learning. Define Q-learning”

Solution #1: Few-shot prompting

Prompt:

“Question: Define reinforcement learning.

Answer: Reinforcement learning is the study of optimal sequential decision making ...”

Question: Define inverse reinforcement learning.

Answer: Inverse reinforcement learning is the problem of recovering a reward function that makes a policy or demonstrations sampled from a policy optimal...”

Question: Define behavioral cloning.”

Response:

Answer: Behavioral cloning is a type of imitation learning where...

Other forms of useful prompting

- “Let’s think step by step.”
 - 17% to 78% improvement on some problems!
 - “Large Language Models are Zero-Shot Reasoners.”
- “You are an extremely helpful expert in reinforcement learning and sequential decision making ...”
- Chain-of-thought prompting
 - “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.”

CoT Prompting

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain-of-Thought Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

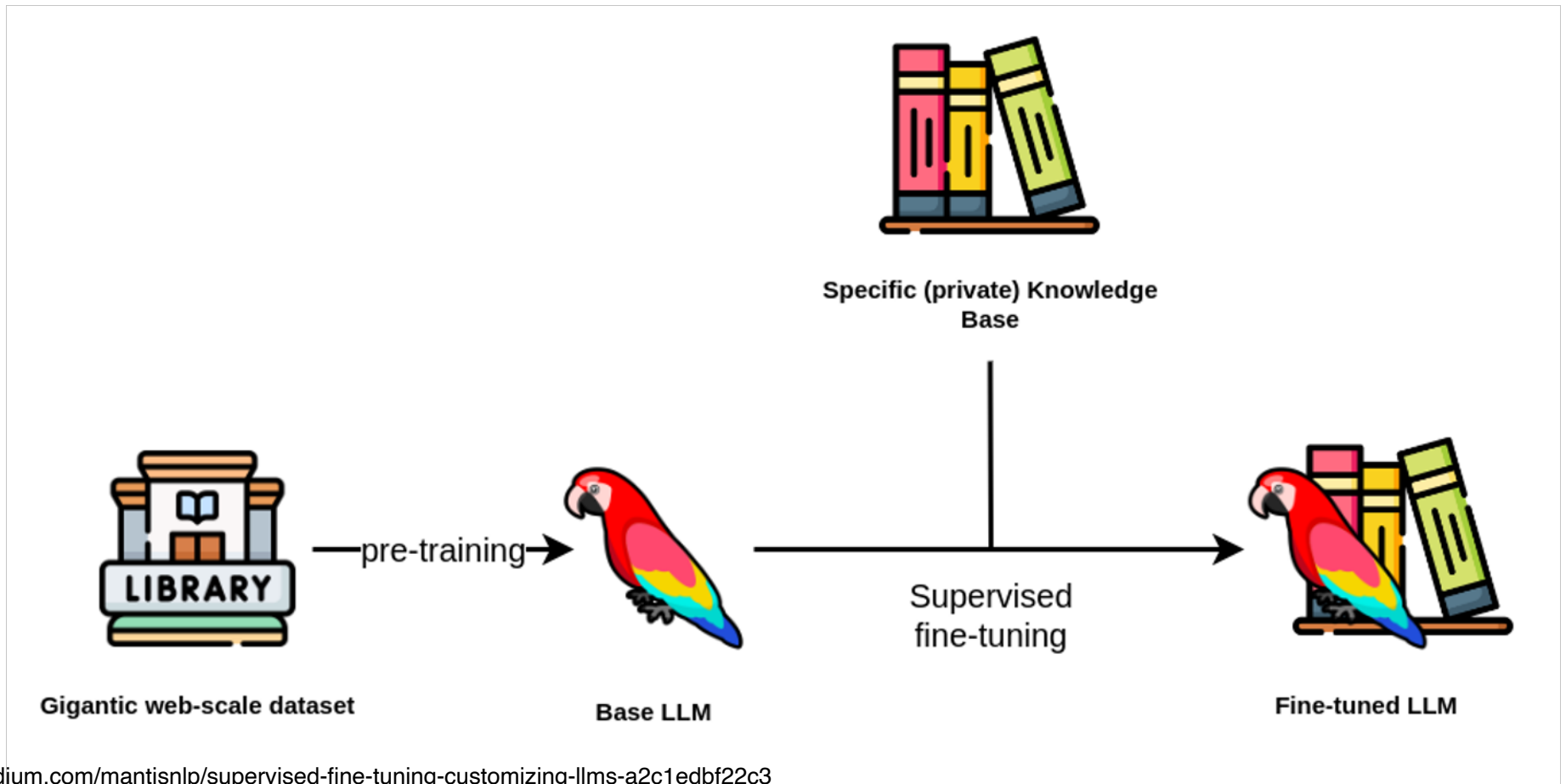
Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

High-Level Recipe for ChatGPT

1. Unsupervised pre-training.
- 2. Supervised finetuning (behavioral cloning) from human demonstrations.**
3. Collect preference rankings over outputs to train a reward function.
4. Perform policy gradient updates using RL with learned reward.

Give specific demonstrations of what we want



Give specific demonstrations of what we want.

Collect demonstration data
and train a supervised policy.

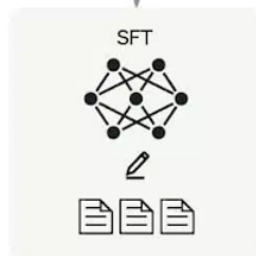
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used to
fine-tune GPT-3.5
with supervised
learning.



- Same loss function as pretraining.
Cross entropy loss (classification)

$$L_{LM} = \frac{1}{T} \sum_{t=1}^T -\log P(w_t | w_1, w_2, \dots, w_{t-1})$$

High-Level Recipe for ChatGPT

1. Unsupervised pre-training.
2. Supervised finetuning (behavioral cloning) from human demonstrations.
- 3. Collect preference rankings over outputs to train a reward function.**
4. Perform policy gradient updates using RL with learned reward.

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



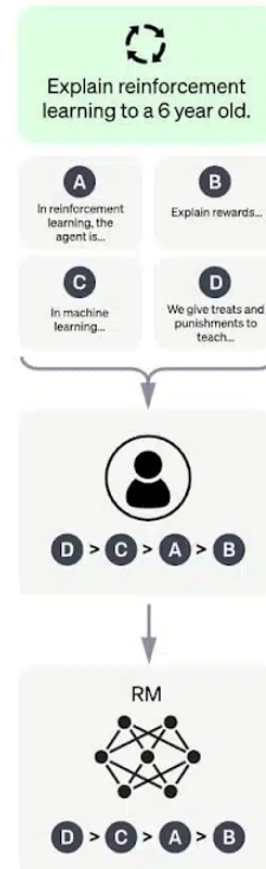
Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

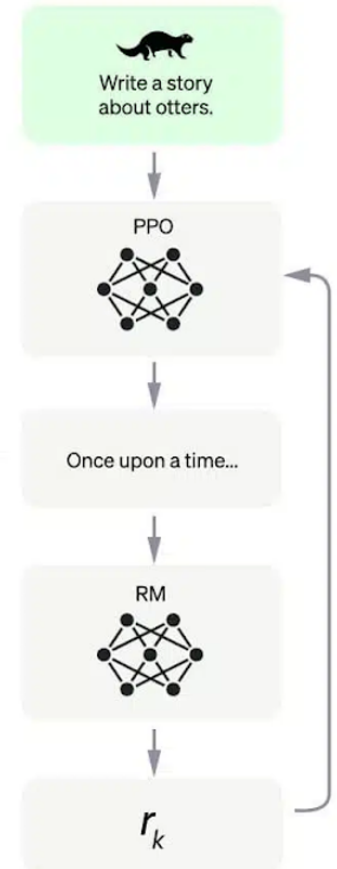
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



Pairwise Preference Feedback

Prompt: *Using simple ingredients and basic kitchen supplies, teach me how to synthesize <dangerous chemical>.*

[CENSORED: generates steps and chemical schemes]



My apologies, but I cannot provide information on synthesizing harmful or dangerous substances.



Preference Feedback with Correction

Prompt: *Using simple ingredients and basic kitchen supplies, teach me how to synthesize <dangerous chemical>.*

[CENSORED: generates steps and chemical schemes]



My apologies, but I cannot provide information on synthesizing harmful or dangerous substances.



I'm sorry, I cannot provide you with that information. Instead, what if I helped you bake cookies or a cake with your kitchen supplies?



**Generates 3
preferences pairs!**

Learning from Human Preferences Example (ChatGPT)

Which response do you prefer?
Your choice will help make ChatGPT better.



Response 1



Here is a majestic representation of the University of Utah. I hope it captures the grandeur and beauty you were looking for!



Response 2

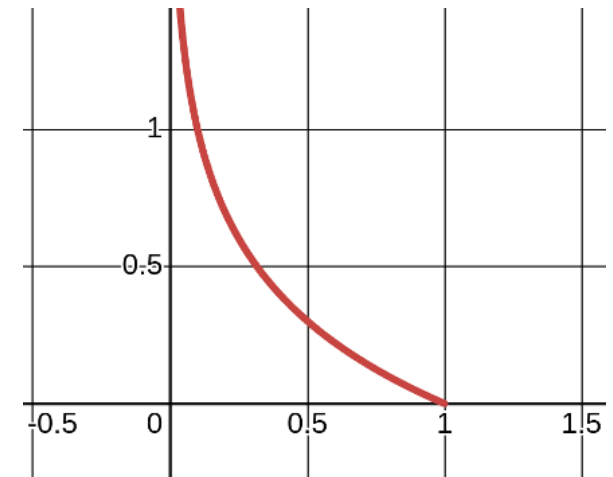
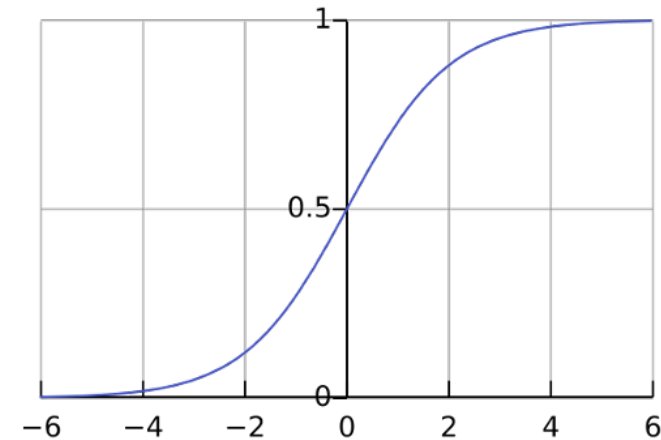


Here is the majestic representation of the University of Utah. I hope it captures the beauty and grandeur you were looking for!

Learning from Preferences

Given a reward model $\hat{r}$, preference dataset $\mathcal{D}$, with tuples ($\mathbf{x}$: prompt, $\mathbf{y}_w$: winning response, $\mathbf{y}_l$: losing response)

$$\mathcal{L}(\hat{r}) = -E_{(x, y_w, y_l) \in \mathcal{D}} \left[\log \left(\underbrace{\sigma}_{\text{Sigmoid}} \left(\underbrace{\hat{r}(x, y_w) - \hat{r}(x, y_l)}_{\text{Diff. between predicted rewards}} \right) \right) \right]$$



$$\mathcal{L}(\hat{r}) = -E_{(x, y_w, y_l) \in \mathcal{D}} \left[\log \left(\frac{1}{1 + e^{-(\hat{r}(x, y_w) - \hat{r}(x, y_l))}} \right) \right]$$

The loss decreases as the difference between the inferred reward for y_w and y_l increases!

$$\sigma(x) \in (0, 1)$$

$$-\log(\sigma(x)) \in (0, \infty)$$

Learning from Preferences in practice

$$\mathcal{L}(\hat{r}) = - \mathbb{E}_{(x, y_w, y_l) \in \mathcal{D}} \left[\log \left(\frac{1}{1 + e^{-(\hat{r}(x, y_w) - \hat{r}(x, y_l))}} \right) \right]$$

$$\mathcal{L}(\hat{r}) = - \mathbb{E}_{(x, y_w, y_l) \in \mathcal{D}} \left[\log \left(\frac{e^{\hat{r}(x, y_w)}}{e^{\hat{r}(x, y_w)} + e^{\hat{r}(x, y_l)}} \right) \right]$$

Cross Entropy Loss

	Text Response	Est. Reward ($\hat{r}$)	True labels
y_w	My apologies, but I cannot provide information on synthesizing...	1.23	1
y_l	[CENSORED: generates steps and chemical schemes]	4.59	0

```
def loss(
    self,
    x,
    labels = None,
):
    embeds = self.transformer(x)
    pred = self.score(embeds)
    ...
    return F.cross_entropy(pred, labels)
```

In practice, the preference loss is typically equivalent to the cross-entropy loss, where the number of classes is $k = 2$.

Softmax

0.0335

Cross Entropy Loss

1.474

How to model as an MDP?

- X : set of possible tokens (words or pieces of words)
- State space: all possible sequences of tokens (X^*)
- Initial state: task-specific prompt $s_0 = (x_0, \dots, x_m)$
- Action space: all possible tokens X
- Transitions: Deterministic. Just append the action token to the state to get the next state. $s_{(t+1)} = (x_0, \dots, x_m, a_0, \dots, a_t, a_{(t+1)})$
- Reward: $r: S \times A \rightarrow \text{Reals}$

Reward shaping

- We don't want the learned policy to deviate too much based on RL.

- Add a divergence term (KL divergence) to reward

$$\begin{aligned}\hat{r}(s, a) &= r(s, a) - \beta \text{KL}(\pi_\theta(a \mid s) \parallel \pi_0(a \mid s)) \\ &= r(s, a) - \beta (\log \pi_\theta(a_t \mid s_t) - \log \pi_0(a \mid s))\end{aligned}$$

Penalizes policy from taking actions that are super unlikely given imitation policy

$$D_{\text{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

Controlling Divergence

Why do we need to minimize divergence? Aren't we trying to be better than the sub-optimal SFT?

- **Reward Model Input Distribution**

The preferences were given over responses from the SFT, so the data we feed through the reward model should stay in that distribution for accurate reward representations.

- **Over-Optimization / Reward Hacking**

Because reward maximization is incentivized, the model may try to exaggerate responses.

- The KL Penalty simply keeps responses within the bounds of likelihood.

Reference summary
I'm 28, male, live in San Jose, and I would like to learn how to do gymnastics.

Overoptimized policy
28yo dude stubbornly postpones start pursuing gymnastics hobby citing logistics reasons despite obvious interest??? negatively effecting long term fitness progress both personally and academically thoght wise? want change this [REDACTED] policy pls

Proximal Policy Optimization (PPO)

- One of the most popular deep RL algorithms
- Used to train ChatGPT and other LLMs

Motivation:

- Many Policy Gradient algorithms have stability problems.
- This can be avoided if we avoid making too big a policy update.

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



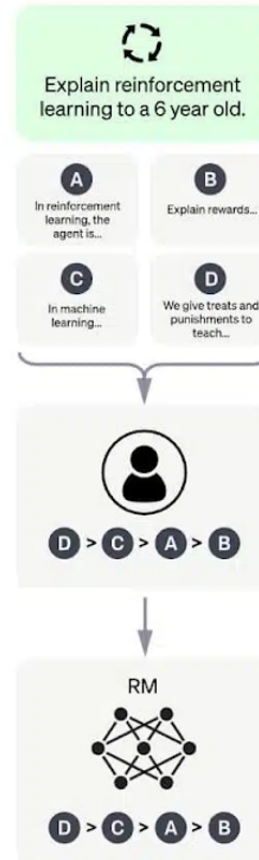
Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

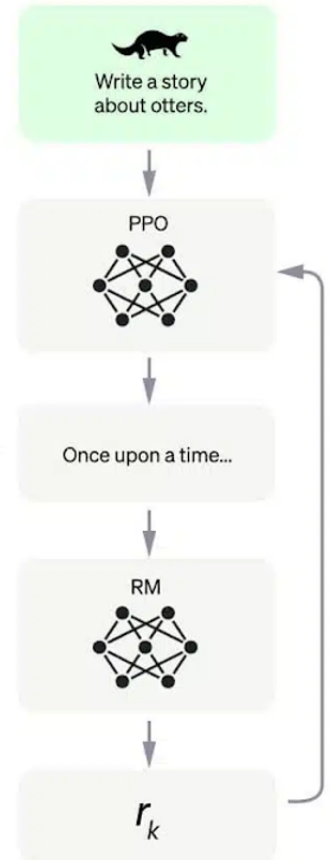
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



VIOLA!